

MAINTAINER-FRIENDLY PDF EDITION

Contract Library

Durable contracts for The Ontology Machine: payloads, files, database surfaces, tool surfaces, and state envelopes.

Core rule: every contract is recorded by producer, consumer, durable shape, source of truth, and known failure modes.

Contents

1. Contract Families	3
2. Contract Source Priority	4
3. Core Invocation Contracts	5
4. Pipeline Artifact Contracts	9
5. Semantic Contracts	18
6. Corpus DB Contracts	22
7. Ontology Contracts	30
8. Agent And Frontend Contracts	33
9. MCP And Kernel Contracts	38
10. Merge And Rebuild Contracts	45
11. Contract Validation Map	49
12. Known Sharp Edges	50
13. Related Chapters	51

Recommended first pass for maintainers: read Contract Families, Source Priority, then the contract family that owns the surface you are changing.

4. Contract Library

This chapter documents the durable contracts that hold The Ontology Machine together. It is not a generated schema dump and it is not a historical change log. It is the handover map for the payloads, files, database surfaces, tool surfaces and state envelopes that other maintainers must not accidentally break.

The most important thing to understand is that not every contract has one neat JSON Schema file. In the current codebase, contracts are distributed across:

- `module-manifest.json` action lists
- `orchestrator_contract` command dataclasses
- `module-local` request validators
- Orchestrator adapter parsers
- Semantic Release builders and validators
- Corpus DB DDL files and read-surface views
- Kernel contract registries and valid fixture files
- MCP tool schema validation
- Client Frontend route handlers and browser-side types

That is not ideal in the abstract, but it is the real implementation. This library therefore records each contract by its producer, consumer, durable shape, source of truth, and known failure modes.

Contract Families

Family	Contract Boundary	Primary Owner
Module invocation	Process-level JSON request/response between Orchestrator and modules	00 - Orchestrator
Pipeline artifact handoff	Files written between Optimizer, Interpreter, Validator, Normalizer and Corpus Builder	00 - Orchestrator
Semantic Release	Taxonomy, projections, release fingerprint and runtime semantic assets	04 - Normalizer
Corpus DB	SQLite schema, materialization, read surfaces, search, Base Graph and ontology tables	05 - Corpus Builder
Agent read/write tools	Query Agent read tools and Ontology Agent write tools	Client Frontend
MCP	Local stdio JSON-RPC tool bridge and permission gate	07 - MCP Server
Kernel workflows	Workflow state, events, interactions, confirmations, recovery and owner adapter calls	08 - Semantic Control Kernel
Frontend runtime	Config, credentials, chat/session/source display and Kernel event polling	Client Frontend

Contract Source Priority

When two files disagree, use this priority order:

1. Executed validators and dispatchers.
2. Action constants, command dataclasses and TypedDict declarations.
3. Active DDL/schema files.
4. Golden fixtures used by tests.
5. `module-manifest.json`.
6. README prose.
7. Older design or migration notes.

This priority matters because a few README lines still lag behind code. For example, Corpus Builder schema version prose in an older README can be stale, while the active schema version is defined in `05 - Corpus Builder/corpus_builder/database/types.py`.

Core Invocation Contracts

Module Manifest Contract

PURPOSE

Each product module declares its public action surface and runtime entry point. The Orchestrator, Edit Suite, MCP Server and test harnesses use these manifests to know which actions exist.

PRODUCER

Each module owns its own `module-manifest.json`.

CONSUMERS

- `00` - Orchestrator
- `06` - Edit Suite
- `07` - MCP Server
- root dev-test tooling
- documentation and handover surfaces

SHAPE

The stable fields are:

```
{
  "module_key": "corpus_builder",
  "display_name": "Corpus Builder Vision",
  "entrypoint": "corpus_builder.orchestrator_contract",
  "actions": ["load_document", "healthcheck"]
}
```

The exact manifest may contain more metadata, but the durable contract is:

- `module_key` identifies the module.
- `display_name` is the human-facing label.
- `actions` is the public action list.
- `entrypoint` or module runtime metadata points to the importable contract module.

CURRENT ACTION SURFACES

Module	Actions
<code>00</code> - Orchestrator	<code>run</code> , <code>reset</code> , <code>reset_pipeline_logs</code> , <code>embeddings</code> , <code>activate_corpus_context</code> , <code>inspect_source_document_sample</code> , <code>kernel_llm_runtime_profile</code> , <code>kernel_llm_generate</code> , <code>healthcheck</code> , <code>create_artifact_tree</code> , <code>validate_artifact_tree</code> , <code>create_pipeline_batch_manifest</code> , <code>finalize_pipeline_batch_manifest</code>
<code>01</code> - Optimizer	<code>classify_document</code> , <code>extract_document</code> , <code>healthcheck</code> , <code>scan_debug_input</code> , <code>debug_run</code>
<code>02</code> - Interpreter	<code>interpret_document</code> , <code>healthcheck</code> , <code>debug_run</code> , <code>generate_llm</code>
<code>03</code> - Validator	<code>validate_document</code> , <code>healthcheck</code> , <code>debug_run</code>
<code>04</code> - Normalizer	<code>normalize_document</code> , <code>build_projection_catalog</code> , <code>build_runtime_semantic_assets</code> , <code>publish_semantic_release</code> , <code>list_default_blueprints</code> , <code>export_default_blueprint_release</code> , <code>create_zero_shot_working_release</code> , <code>healthcheck</code> , <code>debug_run</code>

Module	Actions
05 - Corpus Builder	load_document , activate_semantic_release , activate_corpus_context , create_empty_corpus_db , reset_active_corpus_db , create_and_activate_new_corpus_db , activation_preflight , generate_embeddings , healthcheck , scan_debug_input , debug_run , semantic_status , read_active_semantic_release , load_semantic_release , semantic_audit , backfill_stale , merge_preflight , merge_corpus_databases , validate_artifact_tree , read_database_analysis_evidence , inspect_latest_pipeline_batch , extract_sample_files_for_reingest , restore_pipeline_batch_originals , cleanup_pipeline_batch_materialization , reingest_pipeline_batch , multi_source_merge_preflight , multi_source_merge_databases , write_merge_reconciliation_manifest , backfill_sql_from_merge_artifacts , search , stats , export , preview_rebuild_from_artifacts , rebuild_from_artifacts , create_and_rebuild_new_corpus_db , basic_relation_mining
06 - Edit Suite	healthcheck
07 - MCP Server	serve , healthcheck
08 - Semantic Control Kernel	empty_database_no_semantic_release , empty_database_default_taxonomy_no_projections , empty_database_default_taxonomy_default_projections , empty_database_default_taxonomy_custom_projections , empty_database_custom_taxonomy_no_projections , empty_database_custom_taxonomy_custom_projections , manual_pipeline_run , database_merge_additive_only , database_rebuild_from_artifacts , create_custom_taxonomy_path , create_custom_projection_path , reset_database , kernel_status , kernel_resume_state , kernel_continue_resumable_workflow , kernel_cancel_active_run

SOURCE OF TRUTH

- 00 - Orchestrator/module-manifest.json
- 01 - Optimizer/module-manifest.json
- 02 - Interpreter/module-manifest.json
- 03 - Validator/module-manifest.json
- 04 - Normalizer/module-manifest.json
- 05 - Corpus Builder/module-manifest.json
- 06 - Edit Suite/module-manifest.json
- 07 - MCP Server/module-manifest.json
- 08 - Semantic Control Kernel/module-manifest.json

FAILURE MODES

- Manifest action exists but dispatcher cannot route it.
- Dispatcher action exists but manifest omits it.
- README describes an action that code no longer exposes.
- Tool surfaces copy action names manually and drift.

VALIDATION

Corpus Builder has runtime checks comparing manifest actions against `ACTION_NAMES` and dispatch routes. Other modules mostly rely on their contract tests and direct validators.

Module Process Call Contract

PURPOSE

The Orchestrator invokes headless modules as separate Python process calls and exchanges request/response JSON files with them.

PRODUCER

00 - Orchestrator

CONSUMERS

All orchestrator-bound module contracts.

INVOCATION SHAPE

The active call pattern is:

```
python -m <contract_module> --request request.json --response response.json
```

The module must read the request file, execute the named action, and write a JSON object response file.

MINIMAL REQUEST SHAPE

```
{
  "action": "healthcheck"
}
```

Owner-routed calls may use:

```
{
  "schema_version": "adapter.call_request.v1",
  "owner_action": "validate_artifact_tree",
  "workflow_run_id": "workflow-run-id",
  "adapter_call_id": "adapter-call-id",
  "payload": {}
}
```

MINIMAL RESPONSE SHAPE

```
{
  "status": "ok"
}
```

The exact response fields are action-specific. The durable invariant is that the response is a JSON object, not an array, plain string, log stream or file path.

SOURCE OF TRUTH

- 00 - Orchestrator/orchestrator/integrations/adapter.py
- 00 - Orchestrator/orchestrator/integrations/validation.py
- each module's orchestrator_contract/__init__.py

FAILURE MODES

- response file missing
- response is not valid JSON
- response JSON is not an object
- unknown action or owner_action
- owner envelope fingerprint or target proof mismatch
- process timeout or non-zero exit code

Owner Envelope Contract

PURPOSE

Owner envelopes wrap Kernel-originated owner calls so the owner module can prove that it acted on the exact target the Kernel intended.

PRODUCER

08 - Semantic Control Kernel

CONSUMERS

- 00 - Orchestrator artifact tree actions
- 05 - Corpus Builder Phase 19 owner actions
- MCP bridge when routing Kernel owner calls

REQUEST SHAPE

```
{
  "schema_version": "kernel.pipeline_owner_request.v1",
  "owner_action": "create_artifact_tree",
  "workflow_run_id": "workflow-run-id",
  "adapter_call_id": "adapter-call-id",
  "target_identity": {
    "artifact_root": "C:/path/to/artifact/root"
  },
  "fingerprint": "request-fingerprint",
  "payload": {}
}
```

RESPONSE SHAPE

```
{
  "schema_version": "kernel.pipeline_owner_result.v1",
  "status": "ok",
  "output_refs": {},
  "target_identity_proof": {},
  "receipt_fields": {},
  "diagnostics": [],
  "warnings": []
}
```

INVARIANTS

- `owner_action` must match the allowed owner action for the called module.
- `workflow_run_id` and `adapter_call_id` must be preserved.
- owner responses must include enough proof for Kernel to validate target identity.
- mutating owner failures can be treated as partial-mutation risk if the owner cannot prove a clean outcome.

SOURCE OF TRUTH

- 00 - Orchestrator/orchestrator/workspace_domain/validation.py
- 00 - Orchestrator/orchestrator/workspace_domain/adapter.py
- 05 - Corpus Builder/corpus_builder/orchestrator_contract/validation_owner_envelope.py
- 08 - Semantic Control Kernel/semantic_control_kernel/adapters/owner_response.py

Pipeline Artifact Contracts

Artifact Tree Contract

PURPOSE

The Artifact Tree is the durable folder contract around a Corpus DB. It is the rebuild and evidence surface for ingestion runs, errors, page images, Semantic Releases and logs.

PRODUCER

- `00 - Orchestrator` for tree creation and validation
- `08 - Semantic Control Kernel` when creating workflow targets

CONSUMERS

- `01 - Optimizer`
- `02 - Interpreter`
- `03 - Validator`
- `04 - Normalizer`
- `05 - Corpus Builder`
- `Client Frontend`
- merge and rebuild workflows

SCHEMA VERSION

`kernel_artifact_tree.v1`

FOLDER SHAPE

```
Artifact Root/  
  Input/  
  Corpus/  
  Semantic Release/  
  Documents/  
    logs/  
    normalized/  
    originals/  
    page_images/  
    raw_extracts/  
    requests/  
    structured/  
    validation/  
  Error Cases/
```

INVARIANTS

- `Input` is the intake area for source files.
- `Corpus` contains Corpus DB files.
- `Semantic Release` contains release packages.
- `Documents/page_images` contains rebuild artifacts for page-image truth.
- `document_page_images` inside the DB is still required for evidence back-linking; the folder copy is the artifact-level rebuild surface.
- `Documents/requests` persists LLM/OCR request payloads for inspectability.
- `Error Cases` is the persisted failure evidence surface.

- merge workflows copy `Documents` and `Error Cases` , not live `Corpus` , live `Semantic Release` , or `Input` .

SOURCE OF TRUTH

- `00` - `Orchestrator/orchestrator/workspace_domain/types.py`
- `00` - `Orchestrator/orchestrator/workspace_domain/workflow.py`
- `00` - `Orchestrator/orchestrator/workspace_domain/validation.py`
- `05` - `Corpus Builder/corpus_builder/standalone_artifacts/artifact_tree_contract.py`
- `08` - `Semantic Control Kernel/semantic_control_kernel/workflows/database_creation/artifact_tree_contract.py`

COMMON FAILURE CASES

- path selected outside `ui_state.artifact_folder`
- missing required folder
- target DB path outside `<artifact_root>/Corpus`
- stale merge output copied into `Corpus` instead of preserved as artifact evidence

Pipeline Batch Manifest Contract

PURPOSE

The Pipeline Batch Manifest records a Kernel/Orchestrator pipeline batch so the run can later be inspected, retried, cleaned up, or reingested.

PRODUCER

- `00` - `Orchestrator`
- `08` - `Semantic Control Kernel` when it coordinates pipeline runs

CONSUMERS

- `05` - `Corpus Builder` reingest and cleanup actions
- Kernel pipeline recovery
- Frontend status and final notice surfaces

SHAPE

The manifest is registered as `kernel.pipeline_batch_manifest.v1` in Kernel fixtures and includes workflow/run identifiers, input refs, output refs, document counts and batch state.

INVARIANTS

- It is run-scoped, not document-scoped.
- It must be written before downstream cleanup/reingest actions can rely on it.
- Paths recorded in it must stay inside the chosen Artifact Tree.

SOURCE OF TRUTH

- `00` - `Orchestrator/module-manifest.json`
- `00` - `Orchestrator/orchestrator/orchestrator_contract/types.py`
- `08` - `Semantic Control Kernel/dev-tests/fixtures/contracts/kernel__pipeline_batch_manifest__v1.valid.json`

Per-Document Path Contract

PURPOSE

The Orchestrator creates stable per-document handoff paths for every document or page unit moving through the pipeline.

PRODUCER

00 - Orchestrator

CONSUMERS

- 01 - Optimizer
- 02 - Interpreter
- 03 - Validator
- 04 - Normalizer
- 05 - Corpus Builder

CANONICAL NAMES

```
<stem>.raw.json  
<stem>.structured.json  
<stem>.vision_validation_report.json  
<stem>.files_validation_report.json  
<stem>.structured.normalized.json  
interpreter.request.json  
<stem>.run.log
```

INVARIANTS

- page-scoped artifacts must preserve page identity.
- multi-page source documents can produce several materialized document rows.
- scalar compatibility fields may exist, but documentation and code must not assume a scalar-only one-file one-document path.

SOURCE OF TRUTH

- 00 - Orchestrator/orchestrator/pipeline/document_types.py
- 00 - Orchestrator/orchestrator/pipeline/policy.py
- 00 - Orchestrator/orchestrator/pipeline/request_enrichment.py

Optimizer Raw Payload Contract

PURPOSE

The Optimizer turns source files into non-semantic raw evidence, page images, OCR/request artifacts and JSON payloads for the downstream Interpreter.

PRODUCER

01 - Optimizer

CONSUMER

00 - Orchestrator request enrichment

ACTIONS

- `classify_document`
- `extract_document`
- `scan_debug_input`
- `debug_run`
- `healthcheck`

REQUEST FIELDS

For extraction, the important fields are:

- `source_path`
- `raw_output_path`
- `page_assets_dir`
- optional `ocr_request_dir`
- optional runtime policy/config paths

RESPONSE FIELDS

The extraction response includes:

- `status`
- `content_hash`
- `ingest_id`
- `document_raw_path`
- `page_raw_paths`
- `page_asset_paths`
- `ocr_request_paths`

RAW PAYLOAD INVARIANTS

- The raw payload is evidence, not interpretation.
- Vision and file profiles route differently, but both feed the same downstream request enrichment boundary.
- For request enrichment, current vision raw payloads use `schema_version = optimizer_raw_v2`.
- OCR request payloads for scans/images are persisted in `Documents/requests` when generated.

SOURCE OF TRUTH

- 01 - `Optimizer/ingestion_layer_vision/orchestrator_contract/validation.py`
- 01 - `Optimizer/ingestion_layer_vision/orchestrator_contract/workflow.py`
- 01 - `Optimizer/ingestion_layer_vision/orchestrator_contract/classification_routing.py`
- 01 - `Optimizer/ingestion_layer_file/orchestrator_contract/workflow.py`
- 00 - `Orchestrator/orchestrator/pipeline/request_enrichment_vision.py`
- 00 - `Orchestrator/orchestrator/pipeline/request_enrichment_file.py`

FAILURE MODES

- source path invalid or outside expected root
- unsupported file kind
- OCR/render height or page asset failure

- raw payload has legacy keys rejected by request enrichment
- page asset produced but not linked in raw payload

Interpreter Request Contract

PURPOSE

The Interpreter request is the Orchestrator-owned payload handed to the LLM Interpreter. It is created after Optimizer extraction and before the actual LLM interpretation call.

PRODUCER

00 - Orchestrator request enrichment

CONSUMER

02 - Interpreter

SHAPE

The request contains:

- source
- context
- page_assets
- ocr_reference.blocks
- prompt/runtime enrichment needed by the Interpreter

INVARIANTS

- The request is page-scoped where the source material is page-scoped.
- It must be persisted as `interpreter.request.json`.
- It is the right place for Orchestrator-owned context, not for Interpreter side adaptation of raw Optimizer payloads.

SOURCE OF TRUTH

- 00 - Orchestrator/orchestrator/pipeline/request_enrichment.py
- 00 - Orchestrator/orchestrator/pipeline/request_enrichment_vision.py
- 00 - Orchestrator/orchestrator/pipeline/request_enrichment_file.py
- 02 - Interpreter/llm_interpreter/orchestrator_contract/types.py
- 02 - Interpreter/llm_interpreter/orchestrator_contract/validation.py

FAILURE MODES

- missing request file
- incompatible raw schema
- page asset reference missing
- context omitted for a source type that needs it

Structured Output Contract

PURPOSE

Structured output is the Interpreter result. It captures what the model read from the request before formal validation or semantic normalization.

PRODUCER

02 - Interpreter

CONSUMERS

- 03 - Validator
- 04 - Normalizer
- 05 - Corpus Builder for traceability

REQUEST COMMAND

`InterpretDocumentCommand` requires:

- `request_path`
- `structured_output_path`
- `runtime_settings.model`
- `runtime_settings.max_output_tokens`
- optional `debug_bundle_dir`

RESPONSE SHAPE

```
{
  "status": "ok",
  "structured_path": "Documents/structured/example.structured.json",
  "needs_review": false,
  "review_reason": "",
  "debug_bundle_path": null
}
```

INVARIANTS

- The Interpreter writes structured JSON to the path supplied by the Orchestrator.
- `needs_review` is advisory but must survive into DB materialization.
- `generate_llm` is a separate Kernel-facing helper and returns `kernel.llm_provider_response.v1` output refs.

SOURCE OF TRUTH

- 02 - `Interpreter/llm_interpreter/orchestrator_contract/types.py`
- 02 - `Interpreter/llm_interpreter/orchestrator_contract/workflow.py`
- 02 - `Interpreter/llm_interpreter/orchestrator_contract/generate_action.py`
- 02 - `Interpreter/llm_interpreter/prompts/schema.py`

Validation Report Contract

PURPOSE

The Validator checks structured output and writes a validation report used by Corpus Builder and error/review surfaces.

PRODUCER

03 - Validator

CONSUMERS

- 00 - Orchestrator
- 05 - Corpus Builder
- Edit/debug surfaces

REQUEST COMMAND

`ValidateDocumentCommand` requires:

- `structured_path`
- `validation_output_path`
- optional `raw_path`

RESPONSE SHAPE

```
{
  "status": "ok",
  "report_path": "Documents/validation/example.vision_validation_report.json",
  "needs_review": false,
  "detail": {},
  "error": null
}
```

INVARIANTS

- Legacy request fields such as `output_dir`, `raw_root` and `report_name` are rejected by the active contract.
- The report is a durable artifact, not just a process response.
- Validator review flags are carried into the Corpus DB document row.

SOURCE OF TRUTH

- 03 - Validator/validator_vision/orchestrator_contract/types.py
- 03 - Validator/validator_vision/orchestrator_contract/validation.py
- 03 - Validator/validator_vision/orchestrator_contract/workflow.py

Normalized Payload Contract

PURPOSE

The Normalizer maps structured output into the active Semantic Release and writes the canonical normalized document payload used by Corpus Builder.

PRODUCER

04 - Normalizer

CONSUMERS

- 05 - Corpus Builder
- debug and rebuild flows

REQUEST COMMAND

`NormalizeDocumentCommand` requires:

- `structured_path`
- `normalized_output_path`
- `runtime_settings.model`
- `runtime_settings.max_output_tokens`
- optional `request_output_path`
- optional embedded `release`

NORMALIZED OUTPUT SHAPE

The Normalizer model output follows the template built from:

- `processing`
- `context`
- `content.structure`
- `content.fields`
- `content.rows`
- `content.free_text`

The runtime prompt injects projection-specific field codes, row types, cell codes, promotion rules and compatibility rules.

RESPONSE SHAPE

```
{
  "status": "ok",
  "output_path": "Documents/normalized/example.structured.normalized.json",
  "request_path": "Documents/requests/example.normalizer.request.json",
  "needs_review": false,
  "message": "",
  "review_reason": "",
  "duration_ms": 0
}
```

INVARIANTS

- `content.fields` must use allowed field codes from the active projection.
- `content.rows` must use allowed row types and cell codes.
- promotion-backed fields should be filled explicitly when input evidence exists.
- `cardinality=multi` values are arrays, not comma-separated strings.
- nullable compatibility is allowed; invented values are not.

SOURCE OF TRUTH

- 04 - `Normalizer/normalizer_vision/orchestrator_contract/types.py`

- 04 - Normalizer/normalizer_vision/orchestrator_contract/validation_actions.py
- 04 - Normalizer/normalizer_vision/orchestrator_contract/workflow.py
- 04 - Normalizer/normalizer_vision/prompts/contract_output_schema.py
- 04 - Normalizer/normalizer_vision/prompts/promotion_contract.py

FAILURE MODES

- structured input missing
- projection field code not recognized
- row cell code emitted outside the projection contract
- promotion field collapsed into one comma-separated value
- request snapshot omitted even though downstream debugging needs it

Semantic Contracts

Taxonomy Source Contract

PURPOSE

Taxonomy source packages are the editable source material from which Semantic Releases are built.

PRODUCER

04 - Normalizer

CONSUMERS

- Normalizer release publishing
- Kernel custom taxonomy workflows
- Edit Suite taxonomy surfaces

LOCATION

04 - Normalizer/config/taxonomy_sources

INVARIANTS

- active source packages and exported blueprints are different things.
- active taxonomy source packages must have unique IDs.
- YAML source shape is validated fail-closed.
- a source package can later be turned into a Semantic Release, but it is not itself the active DB release.

SOURCE OF TRUTH

- 04 - Normalizer/normalizer_vision/taxonomy_sources/types.py
- 04 - Normalizer/normalizer_vision/taxonomy_sources/workflow.py
- 04 - Normalizer/normalizer_vision/taxonomy_sources/validation.py
- 04 - Normalizer/normalizer_vision/taxonomy_sources/validation_release.py
- 04 - Normalizer/normalizer_vision/taxonomy_sources/validation_sections.py
- 04 - Normalizer/normalizer_vision/orchestrator_contract/workflow_blueprints.py

Semantic Release Contract

PURPOSE

A Semantic Release is the packaged semantic truth used for normalization and Corpus DB materialization. It binds master taxonomy, projections, materialization version, runtime assets and fingerprint.

PRODUCER

04 - Normalizer

CONSUMERS

- 05 - Corpus Builder
- 08 - Semantic Control Kernel
- Client Frontend status surfaces

SHAPE

```
{
  "schema_version": "semantic_release.v1",
  "release_id": "release-id",
  "release_version": "1.0.0",
  "master_taxonomy_id": "taxonomy-id",
  "master_taxonomy_version": "1.0.0",
  "master_taxonomy_release_id": "taxonomy-release-id",
  "runtime_locale": "en",
  "projection_ids": ["default_projection"],
  "materialization_version": "materialization-version",
  "created_at": "2026-06-13T00:00:00Z",
  "fingerprint": "fingerprint",
  "master_taxonomy": {},
  "projections": [],
  "analysis": {},
  "projection_catalog": {},
  "runtime_semantic_assets": {}
}
```

REQUIRED INVARIANTS

- `fingerprint` identifies the release payload.
- `projection_ids` must match the included projections.
- `projection_catalog` and `runtime_semantic_assets` are required for active snapshots in Corpus Builder.
- Corpus Builder revalidates release keys and fingerprint before activation.
- Semantic Release activation is a Corpus Builder DB operation, not merely copying a JSON file.

SOURCE OF TRUTH

- 04 - Normalizer/normalizer_vision/semantic_release/types.py
- 04 - Normalizer/normalizer_vision/semantic_release/workflow.py
- 04 - Normalizer/normalizer_vision/orchestrator_contract/workflow_release_actions.py
- 05 - Corpus Builder/corpus_builder/semantic_release/types.py
- 05 - Corpus Builder/corpus_builder/semantic_release/validation.py
- 05 - Corpus Builder/corpus_builder/semantic_release/snapshot_identity.py

FAILURE MODES

- release lacks projection catalog
- runtime semantic assets missing
- fingerprint mismatch
- foreign master taxonomy or projection mismatch
- DB active state and installation state drift

Projection Catalog Contract

PURPOSE

The Projection Catalog describes the usable projections inside a Semantic Release and gives Normalizer/Corpus Builder enough metadata to materialize documents deterministically.

PRODUCER

04 - Normalizer

CONSUMERS

- Normalizer prompt builder
- Corpus Builder materialization
- Semantic Release audit and status tools

CORE FIELDS

- `projection_id`
- `projection_family`
- `master_taxonomy_id`
- `master_taxonomy_version`
- `projection_version`
- `projection_fingerprint`
- `materialization_profile_id`

INVARIANTS

- projection IDs are stable within a release.
- projection fingerprints drive stale detection.
- merged releases may preserve source projections rather than forcing one merged generic projection.

SOURCE OF TRUTH

- 05 - `Corpus Builder/corpus_builder/semantic_release/types.py`
- 04 - `Normalizer/normalizer_vision/runtime_semantic_assets/types.py`

Promotion Contract

PURPOSE

Promotion rules define which normalized fields become query-friendly materialized DB facts.

PRODUCER

04 - Normalizer

CONSUMERS

- Normalizer prompt builder
- Corpus Builder materialization
- Query Agent read surfaces

SHAPE

A rule maps a normalized source path to a materialized slot:

```
content.fields.<field_code> -> <slot>
```

The slot carries:

- cardinality
- value_type
- optional query_role

INVARIANTS

- promotion-backed fields are materialization inputs.
- row cells may reference promoted values but do not replace promotion-backed fields.
- multi-cardinality values must be arrays.

SOURCE OF TRUTH

- 04 - Normalizer/normalizer_vision/prompts/promotion_contract.py
- 05 - Corpus Builder/corpus_builder/semantic_release/types.py
- 05 - Corpus Builder/corpus_builder/semantic_release/materialization_domain.py

Corpus DB Contracts

Corpus DB Schema Contract

PURPOSE

The Corpus DB is the materialized query, evidence, ontology and rebuild target surface.

PRODUCER

05 - Corpus Builder

CONSUMERS

- Query Agent
- Ontology Agent
- Kernel rebuild/merge workflows
- Client Frontend image/source viewer
- export/search/stats tools

SCHEMA VERSION

Current code version: 10

TABLE FAMILIES

Family	Tables
Documents	documents, document_payloads, extracted_fields, extracted_rows, relations, tags, people, organizations
Evidence	evidence_atoms, slot_candidates, document_promotions, candidate_evidence
Search	embedding_chunks, embeddings, documents_fts_content, load_history
Materialization	installation_state, semantic_snapshots, document_processing_state, document_entities, entity_attributes, entity_relations, semantic_evidence_links, materialization_runs, materialization_audit
Page Images	document_page_images
Source / Ontology	source_documents, source_document_pages, source_document_classifications, ontology_lenses, ontology_runs, ontology_terms, ontology_nodes, ontology_edges, ontology_assertions, ontology_evidence_links, ontology_activation, ontology_embedding_chunks, ontology_edit_log
Structure	structural_units, structural_unit_relations

READ SURFACE VIEWS

Surface	Example Views
Base	vw_base_evidence_atoms, vw_base_slot_candidates, vw_document_promotions_current, vw_document_header_surface, vw_document_search_surface, vw_observed_semantics, vw_materialized_semantics
Source	vw_source_document_pages, vw_source_document_classifications, vw_source_document_surface, vw_same_source_document_pages, vw_structural_units, vw_structural_unit_relations, vw_source_document_entities, vw_source_document_evidence_atoms, vw_source_document_promotions

Surface	Example Views
Ontology	<code>vw_active_ontology_nodes</code> , <code>vw_active_ontology_edges</code> , <code>vw_active_ontology_assertions</code> , <code>vw_query_surface_with_active_ontology</code>

INVARIANTS

- Schema initialization applies tables, indexes and views together.
- `semantic_snapshots` and `embedding_chunks` may be treated as optional read tables by older initialization checks, but current DDL creates them.
- DB page images are required for evidence back-linking.
- folder page images remain rebuild artifacts.

SOURCE OF TRUTH

- 05 - Corpus Builder/corpus_builder/database/types.py
- 05 - Corpus Builder/corpus_builder/database/workflow.py
- 05 - Corpus Builder/corpus_builder/database/validation.py
- 05 - Corpus Builder/corpus_builder/database/schema_document_tables.py
- 05 - Corpus Builder/corpus_builder/database/schema_evidence.py
- 05 - Corpus Builder/corpus_builder/database/schema_search.py
- 05 - Corpus Builder/corpus_builder/database/schema_materialization.py
- 05 - Corpus Builder/corpus_builder/database/schema_page_images.py
- 05 - Corpus Builder/corpus_builder/database/schema_ontology.py
- 05 - Corpus Builder/corpus_builder/database/schema_structure.py
- 05 - Corpus Builder/corpus_builder/database/schema_read_surface.py

Document Materialization Contract

PURPOSE

Document materialization writes normalized, structured, raw and validation payloads into the Corpus DB and derives searchable fields, rows, evidence, semantic promotions and review flags.

PRODUCER

05 - Corpus Builder

CONSUMERS

- Query Agent
- Ontology Agent
- export/search/stats
- rebuild and merge

INPUT COMMAND

`LoadDocumentCommand` requires:

- `corpus_db_path`
- `normalized_path`
- `structured_path`

- `validation_path`
- optional `raw_path`
- optional `persist_page_images_in_db`
- optional `page_images_dir`

INVARIANTS

- materialization is normalized-first.
- structured/raw/validation payloads are preserved as evidence.
- review flags from Interpreter, Validator and Normalizer survive into `documents`.
- Semantic Release state determines current projection-backed materialization.

SOURCE OF TRUTH

- 05 - `Corpus Builder/corpus_builder/orchestrator_contract/command_types.py`
- 05 - `Corpus Builder/corpus_builder/orchestrator_contract/workflow.py`
- 05 - `Corpus Builder/corpus_builder/loader/document_record.py`
- 05 - `Corpus Builder/corpus_builder/loader/semantic_repository_core.py`

Source Document And Page Contract

PURPOSE

This contract joins page-wise materialized rows back into their source documents.

PRODUCER

- Corpus Builder loader
- deterministic `basic_relation_mining`

CONSUMERS

- Query Agent
- Ontology Agent
- Base Graph views
- document/page count summaries
- multi-page source viewer logic

TABLES

- `source_documents`
- `source_document_pages`
- `source_document_classifications`

IDENTITY RULES

- source page suffixes are one-based.
- DB `page_index` is zero-based.
- `source_document_id` is taken from materialized source identity when available; deterministic Base Graph work should not invent a semantic source identity.
- legacy backfill may infer weak identities from paths, but this is weaker than owner-provided source identity.

COUNTING RULE

For page-wise corpora, page totals come from:

```
SELECT COUNT(*) FROM source_document_pages;
```

or:

```
SELECT COUNT(*) FROM structural_units WHERE unit_type = 'page_unit';
```

Do not sum `documents.page_count` or `documents.source_page_count`, because those source-level values repeat on page rows.

SOURCE OF TRUTH

- 05 - Corpus Builder/corpus_builder/models/source_identity.py
- 05 - Corpus Builder/corpus_builder/loader/document_record.py
- 05 - Corpus Builder/corpus_builder/database/schema_ontology.py
- 05 - Corpus Builder/corpus_builder/database/schema_read_surface_source_views.py

Base Graph Contract

PURPOSE

The Base Graph is the deterministic structural relation layer over page-wise materialization. It creates source-document/page relations, page units and basic classifications without LLM mining.

PRODUCER

05 - Corpus Builder via `basic_relation_mining`

CONSUMERS

- Query Agent
- Ontology Agent
- read-surface views
- future segmentation and ontology work

TABLES

- `source_documents`
- `source_document_pages`
- `relations`
- `source_document_classifications`
- `structural_units`
- `structural_unit_relations`

INVARIANTS

- deterministic only
- no LLM involvement
- base structure only
- `relations` is corpus base graph, not ontology-specific relation truth.

- ontology-specific readings belong in ontology tables and lenses.
- `chapter`, `section` and `page_span` style structural targets can exist as unfilled placeholders; `base_unit` and `page_unit` can be populated.

SOURCE OF TRUTH

- 05 - Corpus Builder/corpus_builder/ontology/basic_relation_mining.py
- 05 - Corpus Builder/corpus_builder/ontology/basic_relation_writes.py
- 05 - Corpus Builder/corpus_builder/ontology/basic_relation_structural.py
- 05 - Corpus Builder/corpus_builder/database/schema_structure.py
- 05 - Corpus Builder/corpus_builder/database/schema_ontology.py

Source Document Classification Contract

PURPOSE

Source-document classification records classification at source-document level without pretending that every page row is an independent document.

PRODUCER

- deterministic Base Graph for `classification_scope = 'base'`
- Semantic Release materialization for `classification_scope = 'semantic_release'`
- Ontology Agent for `classification_scope = 'ontology'`

CONSUMER

- Query Agent summaries
- Ontology Agent analysis
- read-surface views

FIELDS

- `source_document_id`
- `classification_scope`
- `ontology_id`
- `document_type`
- `category`
- `subcategory`
- `confidence`
- `status`
- `basis_json`
- `created_by`

INVARIANTS

- deterministic scopes should mark ambiguous or unresolved cases instead of fabricating a false source-level truth.
- Ontology Agent writes must use `classification_scope = 'ontology'` and an existing `ontology_id`.
- Ontology Agent preflight rejects writes to deterministic `base` or `semantic_release` scopes.

SOURCE OF TRUTH

- 05 - Corpus Builder/corpus_builder/database/schema_ontology.py
- Client Frontend/client_frontend/ontology_agent/write_preflight_refs.js

Materialized Semantics Contract

PURPOSE

Materialized semantics turns normalized projection-backed values into queryable promotions, candidates, entity rows and evidence links.

PRODUCER

05 - Corpus Builder

CONSUMERS

- Query Agent
- Ontology Agent
- read-surface views
- Semantic Release audit/status

TYPED SHAPE

MaterializedSemantics contains:

- projection_id
- projection_fingerprint
- document_promotions
- slot_candidates
- entities
- entity_attributes
- entity_relations
- processing_state
- audits

STALENESS RULE

If the active projection fingerprint differs from the fingerprint recorded in `document_processing_state`, the document is stale for that projection.

SOURCE OF TRUTH

- 05 - Corpus Builder/corpus_builder/semantic_release/types.py
- 05 - Corpus Builder/corpus_builder/semantic_release/workflow.py
- 05 - Corpus Builder/corpus_builder/semantic_release/materialization_domain.py
- 05 - Corpus Builder/corpus_builder/loader/semantic_repository_core.py

Evidence Contract

PURPOSE

Evidence rows preserve the connection between extracted content, promoted values, ontology claims and source material.

PRODUCER

05 - Corpus Builder

CONSUMERS

- Query Agent verification
- Ontology Agent evidence links
- read-surface views

TABLES

- evidence_atoms
- slot_candidates
- document_promotions
- candidate_evidence
- semantic_evidence_links
- ontology_evidence_links

INVARIANTS

- evidence atoms are provenance tables, not the default place to read direct document facts.
- Query Agent should use read-surface views or current promotions for normal answers, then evidence tables for verification.
- ontology evidence links must point to valid existing evidence targets.

SOURCE OF TRUTH

- 05 - Corpus Builder/corpus_builder/database/schema_evidence.py
- 05 - Corpus Builder/corpus_builder/database/schema_materialization.py
- 05 - Corpus Builder/corpus_builder/database/schema_ontology.py
- Client Frontend/client_frontend/ontology_agent/write_preflight_refs.js

Embedding Contract

PURPOSE

Embeddings support semantic search over documents and ontology objects.

PRODUCER

- 05 - Corpus Builder for document/chunk embeddings
- Client Frontend Ontology Agent post-write refresh for ontology embeddings
- 08 - Semantic Control Kernel rebuild/embedding workflows

TABLES

- embedding_chunks

- `embeddings`
- `ontology_embedding_chunks`

INVARIANTS

- document embeddings require embedding runtime settings.
- ontology embedding chunks require `ontology_id`, `object_type` and `object_id`.
- if embedding credentials are missing, ontology writes can remain valid but the Query Agent may not benefit from refreshed ontology search.
- rebuild treats embeddings as optional if unconfigured unless policy requires them.

SOURCE OF TRUTH

- 05 - `Corpus Builder/corpus_builder/database/schema_search.py`
- 05 - `Corpus Builder/corpus_builder/database/schema_ontology.py`
- 05 - `Corpus Builder/corpus_builder/embeddings/types.py`
- 08 - `Semantic Control Kernel/semantic_control_kernel/policy/rebuild_policy.py`
- 08 - `Semantic Control Kernel/semantic_control_kernel/workflows/rebuild/embeddings.py`

Ontology Contracts

Ontology Lens Contract

PURPOSE

An ontology lens is a versionable, evidence-bound reading over the same materialized corpus. It can represent analysis, correction, review, thematic structure, peer-review views, audit views or any other computable semantic interpretation that should not overwrite base facts.

PRODUCER

Ontology Agent

CONSUMERS

- Query Agent
- Ontology Agent
- ontology read-surface views
- future Knowledge Mining workflows

TABLES

- `ontology_lenses`
- `ontology_runs`
- `ontology_terms`
- `ontology_nodes`
- `ontology_edges`
- `ontology_assertions`
- `ontology_evidence_links`
- `ontology_activation`
- `ontology_embedding_chunks`
- `ontology_edit_log`

STATUS VALUES

Active DB constraint uses:

- `draft`
- `ready`
- `archived`

Do not use `active` as a lens status. Activation is represented through `ontology_activation`.

INVARIANTS

- one primary lens may be active as primary.
- multiple lenses can coexist over the same corpus.
- correction/audit/review lenses do not rewrite materialized base facts.
- evidence links bind ontology objects back to documents, source documents, structural units, evidence atoms, promotions, fields, rows or entities.
- embeddings are generated over ontology chunks when possible.

SOURCE OF TRUTH

- 05 - Corpus Builder/corpus_builder/database/schema_ontology.py
- 05 - Corpus Builder/corpus_builder/database/schema_read_surface_ontology_views.py
- Client Frontend/client_frontend/ontology_agent/types.js
- Client Frontend/client_frontend/ontology_agent/repository.js

Ontology Agent Write Contract

PURPOSE

The Ontology Agent can directly edit the ontology/relation layer of the active Corpus DB while preserving hard write boundaries and stable object IDs.

PRODUCER

Client Frontend Ontology Agent

CONSUMER

Corpus DB

TOOLS

- Query Agent read tools except `workbench`, including the compact `get_document_*` document view family
- `basic_relation_mining`
- `sql_batch_execute`

BATCH CONTRACT

- max 50 SQL statements per batch
- write preflight before transaction
- `BEGIN IMMEDIATE` transaction for accepted batches
- post-write ontology validation
- embedding refresh attempt after successful ontology edit
- repair loop can retry repairable batch failures

ALLOWED WRITE SCOPE

The current allowlist includes ontology tables and selected relation/source tables. The intended safe boundary is:

- read all
- write ontology/relation layer
- deterministic base/semantic truth should not be overwritten by the agent

Because some low-level allowlisted tables overlap with Base Graph tables, future hardening should treat the preflight rules as the semantic authority, not merely the table allowlist.

REQUIRED ID DISCIPLINE

The agent must provide stable IDs for:

- ontology lenses
- runs

- terms
- nodes
- edges
- assertions
- evidence links
- embedding chunks
- edit log units where applicable

PREFLIGHT EXAMPLES

- ontology edges are node-to-node, not term-to-node.
- `attributes_json` must be valid JSON where non-null constraints require it.
- `ontology_evidence_links.evidence_link_id` is required.
- source-document classifications written by the agent must use `classification_scope = 'ontology'`.
- evidence refs must point to existing valid targets.

SOURCE OF TRUTH

- `Client Frontend/client_frontend/ontology_agent/sql_write_policy.js`
- `Client Frontend/client_frontend/ontology_agent/repository.js`
- `Client Frontend/client_frontend/ontology_agent/write_preflight_schema.js`
- `Client Frontend/client_frontend/ontology_agent/write_preflight_refs.js`
- `Client Frontend/client_frontend/ontology_agent/workflow.js`
- `Client Frontend/client_frontend/ontology_agent/workflow_repair.js`

FAILURE MODES

- foreign-key failure from wrong insert order
- missing required stable ID
- using lens status `active`
- inserting edge endpoint as ontology term instead of ontology node
- missing JSON default such as `{}`
- evidence link target not found

Agent And Frontend Contracts

Query Agent Tool Contract

PURPOSE

The Query Agent answers questions over the active Corpus DB without mutating it.

PRODUCER

Client Frontend Query Agent

CONSUMER

Configured Corpus DB

TOOLS

- `sql_query`
- `get_document_summary`
- `get_document_ontology_evidence`
- `get_document_rows`
- `get_document_provenance`
- `get_document_full`
- `get_document`
- `get_provenance`
- `semantic_search`
- `database_coverage_snapshot`
- source-document read tools
- ontology-lens read tools
- `workbench`

`get_document_summary`, `get_document_ontology_evidence`, `get_document_rows`, `get_document_provenance`, `get_document_full` and legacy `get_document` are one document-view contract family. They use the same repository owner path. The different tool names are agent-facing escalation handles, not separate truth sources.

Expected escalation:

```
summary
-> ontology_evidence / rows / provenance
-> full / legacy get_document only if compact views are insufficient
```

READ-ONLY BOUNDARY

- SQLite opens with `mode=ro`.
- SQL must be a single `SELECT` or `WITH` statement.
- `workbench` blocks writes, network and process execution.
- Query Agent should consider ontology lenses when present, including correction/audit/review lenses that contradict materialized facts.

SOURCE OF TRUTH

- Client Frontend/client_frontend/min_agent/types.js
- Client Frontend/client_frontend/min_agent/workflow.js
- Client Frontend/client_frontend/min_agent/repository.js
- Client Frontend/client_frontend/min_agent/document_repository.js
- Client Frontend/client_frontend/min_agent/document_view_support.js
- Client Frontend/client_frontend/min_agent/sql_policy.js
- Client Frontend/client_frontend/min_agent/workbench_validation.js
- Client Frontend/client_frontend/frontend_policy/defaults.js

Chat And Source Contract

PURPOSE

The chat/source contract is what lets the UI show cited sources, source cards and page images next to an agent answer.

PRODUCER

- Query Agent
- Ontology Agent
- Taxonomy Agent responses where applicable

CONSUMERS

- Client Frontend browser UI
- page image viewer
- source-list validation cues

PUBLIC SOURCE SHAPE

```
{
  "id": "document-id",
  "source_key": "source-key",
  "title": "Document title",
  "type": "document_type",
  "date": null,
  "actor": null,
  "source_page": 1,
  "source_page_count": 3,
  "page": 1,
  "page_count": 3,
  "source_refs": [],
  "snippet": "short text",
  "image_url": "/api/image/document-id/1",
  "viewer_available": true,
  "file_name": "source.pdf"
}
```

DISPLAY RULE

The answer may carry a `sources` array but the UI display state also depends on whether the answer text references those sources by bracket citation or file-name mention. This is deliberate: it makes suspicious or unresolved source claims visible instead of silently trusting every model sentence.

SOURCE OF TRUTH

- Client Frontend/client_frontend/browser/types/index.ts
- Client Frontend/client_frontend/http/policy.js
- Client Frontend/client_frontend/min_agent/source_repository.js
- Client Frontend/client_frontend/browser/render/source_policy.ts
- Client Frontend/client_frontend/browser/main_app/conversation_workflow.ts

Client Frontend REST Contract

PURPOSE

The local frontend server exposes chat, config, Kernel event, interaction, image and admin routes.

PRODUCER

Client Frontend server

CONSUMERS

- browser app
- local user
- Kernel event polling UI

PRIMARY ROUTES

Route	Method	Purpose
/api/v2/health	GET	runtime health
/api/v2/chat	POST	Query Agent chat
/api/v2/pipeline-manager/chat	POST	Taxonomy Agent chat
/api/v2/ontology-agent/chat	POST	Ontology Agent chat
/api/chat/history	GET	Query history list
/api/chat/new	POST	new Query chat
/api/chat/history/<id>	GET / DELETE	history detail/delete
/api/chat/restore/<id>	POST	restore Query chat
/api/pipeline-manager/history	GET	Taxonomy history list
/api/ontology-agent/history	GET	Ontology history list
/api/v2/pipeline-manager/kernel/events	GET	Kernel client event polling
/api/v2/pipeline-manager/kernel/interactions/<id>/response	POST	submit user interaction
/api/v2/pipeline-manager/kernel/interactions/<id>/cancel	POST	cancel user interaction
/api/v2/pipeline-manager/kernel/reset	POST	reset Kernel runtime bridge state
/api/v2/pipeline-manager/run/cancel	POST	cancel pipeline run
/api/image/<document_id>/<page>	GET	page image serving
/api/admin/update-key	POST	local admin credential update

Config and OAuth routes are under `/config` and `/config/api/*`.

SOURCE OF TRUTH

- `Client Frontend/client_frontend/http/api_workflow.js`
- `Client Frontend/client_frontend/http/chat_routes.js`
- `Client Frontend/client_frontend/http/api_workflow_kernel.js`
- `Client Frontend/client_frontend/http/config_workflow.js`
- `Client Frontend/client_frontend/http/credentials_workflow.js`
- `Client Frontend/client_frontend/browser/api/factory.ts`

Frontend Config And Policy Contract

PURPOSE

Frontend config stores local runtime selection, credentials state, model settings, active Corpus DB path and agent policy.

PRODUCER

Client Frontend config page/server

CONSUMERS

- Query Agent
- Ontology Agent
- Taxonomy Agent
- provider adapters
- browser config UI

STORED SURFACES

- `config.json`
- `frontend_policy.json`
- credential state
- OAuth session state
- chat history DB/state

POLICY TOP-LEVEL KEYS

- `chat_history`
- `memory`
- `model_catalog`
- `min_agent`
- `ontology_agent`

INVARIANTS

- config save writes `config.json` and `frontend_policy.json` together.
- policy validation is strict about top-level and nested keys.
- secret fields are handled separately from ordinary editable fields.

- model names in defaults/provider catalog are local defaults, not proof that a remote provider still serves that exact model.

SOURCE OF TRUTH

- Client Frontend/client_frontend/config/types.js
- Client Frontend/client_frontend/config/workflow.js
- Client Frontend/client_frontend/frontend_policy/types.js
- Client Frontend/client_frontend/frontend_policy/validation.js
- Client Frontend/client_frontend/credentials/types.js
- Client Frontend/shared/provider-catalog.json

MCP And Kernel Contracts

MCP Transport Contract

PURPOSE

The MCP Server is a local stdio JSON-RPC bridge for tools. It is not a domain truth owner.

PRODUCER

07 - MCP Server

CONSUMERS

- Client Frontend Pipeline/Taxonomy Agent
- local MCP clients

PROTOCOL

- JSON-RPC 2.0
- Content-Length framing
- MCP protocol version 2024-11-05
- supports `initialize`, `ping`, `tools/list`, `tools/call`

TOOL CATALOG RULES

- tool schemas are closed object schemas
- only `agent_visible` tools are externally listed
- unknown or unclassified tools are rejected

SOURCE OF TRUTH

- 07 - MCP Server/mcp_server/protocol.py
- 07 - MCP Server/mcp_server/server.py
- 07 - MCP Server/mcp_server/tool_catalog.py
- 07 - MCP Server/mcp_server/tool_schema_validation.py
- 07 - MCP Server/mcp_server/tool_visibility.py

MCP Permission Contract

PURPOSE

The permission layer controls which MCP tools are visible/callable at each agent level.

LEVELS

- `L0_READONLY`
- `L1_AUTHOR`
- `L2_OPERATOR`
- `L3_ADMIN`

DEFAULTS

- default level is `L1_AUTHOR`

- environment overrides: `VISION_MCP_AGENT_LEVEL`, `MCP_AGENT_LEVEL`
- fail closed on unclassified tools

IMPORTANT DISTINCTION

The permission level is an agent-surface permission, not a guarantee that a tool is non-mutating. Some workflow tools are listed under broad read/author surfaces but may start creation, merge, reset or pipeline workflows. Runtime semantics still belong to the Kernel.

SOURCE OF TRUTH

- 07 - MCP Server/config/agent_permissions.json
- 07 - MCP Server/mcp_server/permission_defaults.py
- 07 - MCP Server/mcp_server/permissions.py

Kernel MCP Envelope Contract

PURPOSE

Kernel MCP envelopes define how MCP calls enter the Semantic Control Kernel and how results return.

SCHEMAS

- `semantic_control_kernel.mcp_request.v1`
- `semantic_control_kernel.mcp_response.v1`
- `semantic_control_kernel.mcp_tool_definition_list.v1`

VALID RESPONSE STATUSES

- `accepted`
- `running`
- `waiting_for_user`
- `completed`
- `blocked`
- `recovery_required`
- `failed`
- `rejected`

SOURCE OF TRUTH

- 08 - Semantic Control Kernel/semantic_control_kernel/types/mcp.py
- 08 - Semantic Control Kernel/semantic_control_kernel/validation/mcp_validation.py
- 08 - Semantic Control Kernel/semantic_control_kernel/mcp_contract.py

Kernel Permanent Agent Tool Contract

PURPOSE

Permanent Kernel tools are the stable Taxonomy Agent facing workflow surface. The agent chooses a workflow; the Kernel owns execution, state and owner adapter calls.

PUBLIC TOOLS

- `empty_database_no_semantic_release`
- `empty_database_default_taxonomy_no_projections`
- `empty_database_default_taxonomy_default_projections`
- `empty_database_default_taxonomy_custom_projections`
- `empty_database_custom_taxonomy_no_projections`
- `empty_database_custom_taxonomy_custom_projections`
- `manual_pipeline_run`
- `database_merge_additive_only`
- `database_rebuild_from_artifacts`
- `create_custom_taxonomy_path`
- `create_custom_projection_path`
- `reset_database`
- `kernel_status`
- `kernel_resume_state`
- `kernel_continue_resumable_workflow`
- `kernel_cancel_active_run`

AGENT BOUNDARY

- model-authored paths, IDs, permissions and recovery fields are rejected.
- most permanent workflow tools expose an empty model-visible schema.
- `kernel_continue_resumable_workflow` accepts only opaque `resume_option_ref`.
- the dispatcher/runtime path is the behavioral source of truth when old handler-status labels still mention phase-only surfaces.

SOURCE OF TRUTH

- 08 - Semantic Control Kernel/semantic_control_kernel/surface/agent_tools.py
- 08 - Semantic Control Kernel/semantic_control_kernel/surface/mcp_tool_schemas.py
- 08 - Semantic Control Kernel/semantic_control_kernel/services/agent_workflow_dispatcher.py
- 08 - Semantic Control Kernel/semantic_control_kernel/types/agent_tool_constants.py
- Client Frontend/client_frontend/pipeline_agent/kernel_tool_surface.js

Kernel Event And State Contract

PURPOSE

Kernel state records workflow runs, progress, interactions, confirmations, locks, recovery events, mirror events, resume options and receipts.

PRODUCER

08 - Semantic Control Kernel

CONSUMERS

- Client Frontend Taxonomy Agent UI
- MCP host bridge tools

- recovery workflows
- support bundles

STATE ROOT

Default:

```
08 - Semantic Control Kernel/state
```

Override:

```
VISION_KERNEL_STATE_ROOT
```

STATE LAYOUT

- workflow_runs/active/*.json
- workflow_runs/history/*.json
- resume/*.resume.json
- pending_interactions/active/*.json
- pending_interactions/history/*.json
- pending_confirmations/active/*.json
- pending_confirmations/history/*.json
- locks/active/*.json
- locks/history/*.json
- events/progress/<workflow_run_id>/*.json
- events/mirror/*.json
- events/recovery/<recovery_event_id>/
- events/tool_availability/*.json
- receipts/index

EXPLICIT NON-OWNERSHIP

Kernel state does not own:

- Corpus DBs
- Artifact Trees
- Semantic Release packages
- source documents
- Pipeline artifacts
- MCP Server state
- Client Frontend state

REGISTERED CONTRACT IDS

Kernel registers contract IDs for:

- active database state
- database artifact binding
- semantic release attach state
- pipeline batch manifest

- merge manifests
- rebuild manifest
- interaction request/response
- client frontend events
- operation receipts
- locks
- workflow resume/options
- progress events
- mirror events
- recovery options/receipts

SOURCE OF TRUTH

- 08 - Semantic Control Kernel/semantic_control_kernel/types/registry.py
- 08 - Semantic Control Kernel/semantic_control_kernel/repository/state_path_layout.py
- 08 - Semantic Control Kernel/semantic_control_kernel/repository/run_store.py
- 08 - Semantic Control Kernel/semantic_control_kernel/repository/resume_store.py
- 08 - Semantic Control Kernel/semantic_control_kernel/repository/event_store.py
- 08 - Semantic Control Kernel/semantic_control_kernel/repository/recovery_events.py
- 08 - Semantic Control Kernel/dev-tests/fixtures/contracts/*.valid.json

Kernel User Interaction Contract

PURPOSE

User interaction contracts let a workflow pause for a concrete decision and resume safely after the Client Frontend submits a response.

PRODUCER

08 - Semantic Control Kernel

CONSUMERS

- Client Frontend Kernel interaction routes
- Taxonomy Agent UI
- recovery/confirmation workflows

ROUTES

- GET /api/v2/pipeline-manager/kernel/events
- POST /api/v2/pipeline-manager/kernel/interactions/<id>/response
- POST /api/v2/pipeline-manager/kernel/interactions/<id>/cancel

INVARIANTS

- interaction response ID must match route ID.
- response and cancel payloads have distinct allowed statuses.
- completion can auto-render a final notice even if the next poll has no new event.

SOURCE OF TRUTH

- 08 - Semantic Control Kernel/dev-tests/fixtures/contracts/kernel__user_interaction_request__v1.valid.json
- 08 - Semantic Control Kernel/dev-tests/fixtures/contracts/kernel__user_interaction_response__v1.valid.json
- Client Frontend/client_frontend/http/api_workflow_kernel.js
- Client Frontend/client_frontend/http/api_workflow_kernel_validation.js

Kernel Recovery Tool Contract

PURPOSE

Event-scoped recovery tools are only available when Kernel state proves that a specific recovery option is active for a specific event/snapshot.

TOOLS

- kernel_apply_recovery_option
- kernel_open_recovery_dialog
- kernel_retry_recoverable_workflow
- kernel_resolve_stale_lock
- kernel_rebind_database_artifact_tree
- kernel_discard_or_archive_staged_work
- kernel_reconcile_partial_pipeline_run
- kernel_open_support_bundle

INVARIANTS

- event-scoped tools require mirror/recovery/snapshot/client/nonce scope.
- availability is state-confirmed, not model-declared.
- host-only bridge tools are not Agent tools and require VISION_MCP_HOST_BRIDGE_TOKEN.

SOURCE OF TRUTH

- 07 - MCP Server/mcp_server/semantic_control_kernel_tool_scopes.py
- 08 - Semantic Control Kernel/semantic_control_kernel/mcp_event_scope.py
- 08 - Semantic Control Kernel/semantic_control_kernel/surface/event_scoped_tools.py
- 08 - Semantic Control Kernel/semantic_control_kernel/surface/recovery_tools.py

Owner Adapter Boundary Contract

PURPOSE

The Kernel executes real work by calling owner modules and then validating their response, target proof and mutation certainty.

PRODUCER

- 08 - Semantic Control Kernel

CONSUMERS

- owner modules
- Kernel state/recovery
- support bundle diagnostics

INVARIANTS

- owner processes get minimal environment.
- adapter call bundles are recorded under `adapter_calls/<call_id>/`.
- owner status is normalized by Kernel.
- target identity proof is validated.
- uncertain mutating failures are partial-mutation risks.

SOURCE OF TRUTH

- 08 - Semantic Control Kernel/semantic_control_kernel/adapters/invocation.py
- 08 - Semantic Control Kernel/semantic_control_kernel/adapters/owner_response.py
- 08 - Semantic Control Kernel/semantic_control_kernel/debug/adapter_diagnostics.py

Merge And Rebuild Contracts

Database Merge Selection Contract

PURPOSE

The merge selection contract describes which source DBs and Artifact Trees are merged into which target Artifact Tree and target DB.

PRODUCER

08 - Semantic Control Kernel

CONSUMERS

- 05 - Corpus Builder
- Kernel merge workflow
- reconciliation tools

SCHEMA

kernel.database_merge_selection.v1

REQUIRED SELECTION FIELDS

- schema_version
- merge_run_id
- created_at
- selected_by_interaction_id
- source_databases
- target_artifact_root
- target_database_path
- merge_route
- projection_merge_mode
- selection_fingerprint

REQUIRED SOURCE FIELDS

- source_database_id
- source_database_path
- source_artifact_root
- source_state
- source_semantic_release_id
- source_semantic_release_version
- source_release_fingerprint
- source_database_fingerprint
- source_artifact_tree_fingerprint
- source_identity_origin

PROJECTION MERGE MODES

- preserve_source_projections

- `merge_to_single_projection`

Default is `preserve_source_projections`.

INVARIANTS

- target DB must stay under `<target_artifact_root>/Corpus`.
- merge fingerprints differ by producer: Kernel stable path hashes and Corpus Builder SHA digests are not the same fingerprint class.
- filled DB merge must preserve Base Graph and ontology content.
- additive merge should copy SQL data into an empty target DB, then reconcile release/materialization state.

SOURCE OF TRUTH

- 08 - Semantic Control Kernel/semantic_control_kernel/types/merge_contract_fields.py
- 08 - Semantic Control Kernel/semantic_control_kernel/workflows/merge/source_selection_target.py
- 08 - Semantic Control Kernel/dev-tests/fixtures/contracts/kernel_database_merge_selection_v1.valid.json
- 05 - Corpus Builder/corpus_builder/semantic_release/multi_source_merge_validation.py
- 05 - Corpus Builder/corpus_builder/semantic_release/multi_source_merge_preflight.py
- 05 - Corpus Builder/corpus_builder/semantic_release/multi_source_merge_sql_copy.py
- 05 - Corpus Builder/corpus_builder/semantic_release/multi_source_merge_sql_base_graph.py
- 05 - Corpus Builder/corpus_builder/semantic_release/multi_source_merge_sql_ontology.py

Merge Collision And Reconciliation Contract

PURPOSE

Collision and reconciliation manifests make merge ambiguity explicit instead of silently overwriting source truth.

SCHEMAS

- `kernel.database_merge_collision_manifest.v1`
- `kernel.database_merge_id_map.v1`
- `kernel.database_merge_reconciliation_receipt.v1`

REQUIRED COLLISION FIELDS

- `collision_id`
- `collision_class`
- `source_refs`
- `target_ref`
- `default_policy`
- `resolution_owner`
- `resolution_status`
- `selected_resolution`
- `requires_user_choice`
- `blocks_activation`

- `diagnostics`

REQUIRED ID MAP FIELDS

- source and target database IDs
- source and target document IDs
- source and target artifact paths
- source and target pipeline batch IDs
- source and target embedding IDs
- release/taxonomy/projection fingerprints

SOURCE OF TRUTH

- `08 - Semantic Control Kernel/semantic_control_kernel/types/merge_contract_fields.py`
- `08 - Semantic Control Kernel/dev-tests/fixtures/contracts/kernel__database_merge_collision_manifest__v1.valid.json`
- `08 - Semantic Control Kernel/dev-tests/fixtures/contracts/kernel__database_merge_id_map__v1.valid.json`
- `08 - Semantic Control Kernel/dev-tests/fixtures/contracts/kernel__database_merge_reconciliation_receipt__v1.valid.json`
- `05 - Corpus Builder/corpus_builder/semantic_release/multi_source_merge_manifests.py`

Database Rebuild Manifest Contract

PURPOSE

The rebuild manifest records a rebuild from Artifact Tree files into a Corpus DB.

PRODUCER

`08 - Semantic Control Kernel`

CONSUMER

`05 - Corpus Builder`

SCHEMA

`kernel.database_rebuild_manifest.v1`

LOCATION

`Documents/logs/rebuild_runs/<rebuild_run_id>/rebuild_manifest.json`

INVARIANTS

- rebuild requires `Semantic Release/releases/*/release.json`.
- Corpus Builder is called with exact `artifact_root`, `corpus_db_path`, loaded release and `replace_existing = true`.
- embeddings can be regenerated when settings allow it.

SOURCE OF TRUTH

- `08 - Semantic Control Kernel/semantic_control_kernel/types/rebuild.py`

- 08 - Semantic Control Kernel/semantic_control_kernel/workflows/rebuild/manifest.py
- 08 - Semantic Control Kernel/semantic_control_kernel/validation/rebuild_validation.py
- 08 - Semantic Control Kernel/semantic_control_kernel/workflows/rebuild/corpus_rebuild.py
- 08 - Semantic Control Kernel/dev-tests/fixtures/contracts/
kernel__database_rebuild_manifest__v1.valid.json

Contract Validation Map

Contract	Primary Validation
module manifests	module tests, runtime reports, action constants
module process request/response	Orchestrator adapter validation
owner envelopes	owner envelope validators and Kernel owner response parser
Artifact Tree	Orchestrator workspace validation, Kernel artifact tree tests
Optimizer raw	Optimizer contract validation, request enrichment schema checks
Interpreter request	Interpreter contract validation
structured output	Interpreter schema and Validator checks
validation report	Validator contract validation
normalized payload	Normalizer prompt/output contract and Corpus Builder loader
Semantic Release	Normalizer release builder, Corpus Builder release validation
Corpus DB	Corpus Builder DDL validation and schema tests
Base Graph	<code>basic_relation_mining</code> deterministic reports and DB views
ontology writes	Ontology Agent SQL allowlist, preflight, post-write validation
Query Agent tools	read-only SQLite mode, SQL policy, workbench policy
MCP tools	MCP schema validation and permission gate
Kernel contracts	registry, valid/invalid fixture tests, state stores
merge	Kernel selection contract, Corpus Builder merge validation
rebuild	Kernel rebuild manifest and Corpus Builder rebuild action
frontend config	config/policy validators

Known Sharp Edges

Distributed Schemas

Many contracts are enforced by several small files instead of one schema. This is workable, but maintainers must update validators, dataclasses, manifests and tests together.

Response Status Casing

Some older response surfaces use `ok/error`, others use `OK/ERROR` or `PASS/WARN/FAIL`. The Orchestrator parser is permissive. New contracts should prefer lower-case stable status values and document exceptions explicitly.

Multi-Page Scalar Compatibility

Some payloads keep scalar compatibility fields next to arrays. Do not document or implement new behavior as scalar-only. The source-document/page and Base Graph layers are the durable solution for multi-page corpora.

Fingerprint Classes

Not every fingerprint is the same kind of fingerprint. Kernel path hashing and Corpus Builder SHA payload digests serve different purposes. Do not compare them as if they were one universal hash type.

Agent Permission Versus Mutation

MCP permission labels describe tool visibility and callability for agents. They do not by themselves prove that a workflow cannot mutate state. Runtime semantics still belong to Kernel and owner modules.

Legacy Kernel Tools

Do not revive or document these as active tools:

- `llm_action_catalog`
- `open_workflow`
- `execute_readonly_workflow_action`
- `execute_author_workflow_action`
- `execute_operator_workflow_action`
- `execute_admin_workflow_action`
- `interrupt_workflow`
- `close_workflow`

They are intentionally retired/hidden.

Related Chapters

- `01_System_Overview.md` explains why these contracts exist.
- `02_Architecture_Map.md` explains where the boundaries sit.
- `03_Module_Catalog.md` explains which module owns which surface.
- `05_Data_Model.md` should expand the Corpus DB tables and views in more detail.
- `06_Kernel_Workflows.md` should expand the Kernel workflow lifecycle.
- `07_Agent_Surfaces.md` should expand Query, Ontology and Taxonomy Agent behavior.